Built-in Operators in C

Complete MCQ Practice Book

For: OAVS TGT Computer Science Exam	**Questions:** 275 MCQs	**Sections:** 8 Topics

#	Topic	Range	Count
1	Arithmetic Operators	Q1–Q40	40 Qs
2	Relational Operators	Q41–Q80	40 Qs
3	Logical Operators	Q81–Q120	40 Qs
4	Assignment Operators	Q121–Q155	35 Qs
5	Bitwise Operators	Q156–Q195	40 Qs
6	Increment / Decrement	Q196–Q225	30 Qs
7	Conditional Operator	Q226–Q250	25 Qs
8	sizeof Operator	Q251–Q275	25 Qs

Output-Based Questions	Conceptual MCQs	Precedence Problems	Bitwise Traps	Answer Key + Explanations

How to Use This Book

Scope	This book covers all built-in operators in the C programming language, aligned with the OAVS TGT Computer Science syllabus.
Question Types	The book includes output-based questions (predict output of a code snippet), conceptual questions (theory/rules), precedence and associativity problems, bitwise calculation problems, and edge-case/trap questions.
Answer Key	A consolidated answer key is provided at the end of the book for quick reference.
Explanations	Each question is followed by a detailed explanation to reinforce understanding.
Undefined Behaviour	Several questions involve undefined behaviour (UB) in C. These are marked clearly in explanations and are important for competitive exams.
Platform Notes	sizeof results may vary; answers assume a typical 32/64-bit system with 4-byte int unless stated otherwise.

Quick Precedence Reference (High → Low)

Level	Operators	Associativity
1 (Highest)	`() [] -> . post++ post--`	Left → Right
2	`pre++ pre-- + - ! ~ (type) * & sizeof`	Right → Left
3	`* / %`	Left → Right
4	`+ -`	Left → Right
5	`<< >>`	Left → Right
6	`< <= > >=`	Left → Right
7	`== !=`	Left → Right
8	`& (bitwise)`	Left → Right
9	`^`	Left → Right
10	`|`	Left → Right
11	`&&`	Left → Right
12	`||`	Left → Right
13	`?:`	Right → Left
14	`= += -= *= /= %= &= ^= |= <<= >>=`	Right → Left
15 (Lowest)	`,`	Left → Right

Section 1: Arithmetic Operators

Q1. Which symbol represents the modulus operator in C?

A) %
B) //
C) mod
D) %%

■ Correct Answer: A) %
Explanation: % is the modulus operator; it returns the remainder of integer division.

Q2. What is the output of: printf("%d", 10 % 3);

A) 3
B) 1
C) 0
D) 2

■ Correct Answer: B) 1
Explanation: 10 / 3 = 3 remainder 1, so 10 % 3 = 1.

Q3. What is the result of 7 / 2 in C when both operands are int?

A) 3.5
B) 3
C) 4
D) 2

■ Correct Answer: B) 3
Explanation: Integer division truncates towards zero; 7/2 = 3.

Q4. What does the expression 5 + 3 * 2 evaluate to in C?

A) 16
B) 11
C) 10
D) 13

■ Correct Answer: B) 11
Explanation: * has higher precedence than +; 3*2=6, then 5+6=11.

Q5. Which of the following is NOT an arithmetic operator in C?

A) +
B) **
C) %
D) /

■ Correct Answer: B) **
Explanation: C does not have ** for exponentiation; use pow() from math.h.

Q6. What is the output?

```
int a=15, b=4;
printf("%d", a%b);
```

A) 3
B) 1
C) 2
D) 3.75

■ Correct Answer: A) 3
Explanation: 15/4 = 3 remainder 3, so 15%4 = 3.

Q7. What is the result of -7 % 3 in C (C99 and later)?

A) -1
B) 2
C) 1
D) -2

■ Correct Answer: A) -1
Explanation: In C99+, result of % has the sign of the dividend; -7%3 = -1.

Q8. int x = 5; What is x * x + x / x - x % x?

A) 25
B) 26
C) 24
D) 30

■ Correct Answer: B) 26
Explanation: 5*5=25, 5/5=1, 5%5=0 → 25+1-0 = 26.

Q9. What is the value of (int)(7.9) + (int)(2.1) in C?

A) 10
B) 9
C) 7
D) 11

■ Correct Answer: B) 9
Explanation: Casting truncates: (int)7.9=7 and (int)2.1=2; 7+2=9.

Q10. Which operator has highest precedence among +, -, *, /?

A) + and -
B) * and /
C) All equal
D) /

■ Correct Answer: B) * and /
Explanation: Multiplication and division (*,/) have higher precedence than addition/subtraction (+,-).

Q11. What is the output?

```
float a = 5/2;
printf("%.1f", a);
```

A) 2.5
B) 2.0
C) 2
D) 3.0

■ Correct Answer: B) 2.0
Explanation: 5/2 is integer division (=2); storing 2 in float gives 2.0.

Q12. What is the output?

```
float a = 5.0/2;
printf("%.1f", a);
```

A) 2.0
B) 2.5
C) 3.0
D) Error

■ Correct Answer: B) 2.5
Explanation: 5.0/2 promotes 2 to double; result is 2.5.

Q13. What is the result of 2 + 3 * 4 - 6 / 3?

A) 12
B) 12
C) 12
D) All same

> ■ **Correct Answer: A) 12**
> **Explanation:** Precedence: 3*4=12, 6/3=2 → 2+12-2 = 12.

Q14. What is the value of 100 % 10?

A) 10
B) 0
C) 1
D) 100

> ■ **Correct Answer: B) 0**
> **Explanation:** 100 is exactly divisible by 10; remainder = 0.

Q15. int a=7, b=0; What happens when you compute a%b?

A) 0
B) 7
C) Undefined behaviour/crash
D) Compile error

> ■ **Correct Answer: C) Undefined behaviour/crash**
> **Explanation:** Modulo by zero causes undefined behaviour (often a runtime crash).

Q16. What is the output?

```
int x=1000, y=3;
printf("%d", x/y);
```

A) 333
B) 334
C) 333.3
D) 0

> ■ **Correct Answer: A) 333**
> **Explanation:** Integer division: 1000/3 = 333 (truncated).

Q17. Which expression correctly computes a to the power b in C?

A) a**b
B) `pow(a,b)`
C) a^b
D) a^^b

> ■ **Correct Answer: B) pow(a,b)**
> **Explanation:** pow(a,b) from <math.h> computes a raised to power b.

Q18. What is the output?

```
printf("%d", 25 % 5);
```

A) 5
B) 1
C) 0
D) 25

> ■ **Correct Answer: C) 0**
> **Explanation:** 25 is exactly divisible by 5; remainder = 0.

Q19. int a=3, b=4, c=5; What is a*b+c%a?

A) 14
B) 13
C) 12
D) 15

> ■ **Correct Answer: A) 14**
> **Explanation:** * and % first: 3*4=12, 5%3=2 → 12+2=14.

Q20. Which statement about arithmetic operators in C is TRUE?

A) % works on float
B) / always returns float
C) Both operands of % must be integers
D) + can only add numbers

> ■ **Correct Answer: C) Both operands of % must be integers**
> **Explanation:** The modulus operator % is defined only for integer operands in C.

Q21. What is the output?

```
int a=10;
printf("%d", a/3*3);
```

A) 10
B) 9
C) 3
D) 1

■ Correct Answer: B) 9

Explanation: Left-to-right: 10/3=3 (integer), then 3*3=9.

Q22. What is the result of (5+3)*(10-4)/2?

A) 24
B) 16
C) 32
D) 14

■ Correct Answer: A) 24

Explanation: Brackets first: (5+3)=8, (10-4)=6 → 8*6=48, 48/2=24.

Q23. What is 17 % 5 in C?

A) 3
B) 2
C) 4
D) 1

■ Correct Answer: B) 2

Explanation: 17/5 = 3 remainder 2, so 17%5 = 2.

Q24. What is the output?

```
int x=5,y=2;
printf("%d %d",x/y,x%y);
```

A) 2 1
B) 2 0
C) 3 1
D) 2.5 1

■ Correct Answer: A) 2 1

Explanation: 5/2=2 (integer), 5%2=1.

Q25. In C, what is the associativity of arithmetic operators * and /?

A) Right to Left
B) Left to Right
C) Non-associative
D) Depends on compiler

■ Correct Answer: B) Left to Right
Explanation: *, / and % are left-associative; evaluated left to right when at same level.

Q26. What is 0 % 5 in C?

A) 5
B) 1
C) 0
D) Undefined

■ Correct Answer: C) 0
Explanation: 0 divided by 5 has remainder 0.

Q27. What is the output?

```
printf("%d", 2+3*4-8/4);
```

A) 12
B) 10
C) 14
D) 8

■ Correct Answer: A) 12
Explanation: 3*4=12, 8/4=2 → 2+12-2=12.

Q28. Which operation can cause overflow in int arithmetic in C?

A) Adding two small ints
B) Multiplying INT_MAX by 2
C) Subtracting zero
D) Dividing by 2

■ Correct Answer: B) Multiplying INT_MAX by 2
Explanation: INT_MAX * 2 exceeds int range causing signed integer overflow (undefined behaviour).

Q29. What is the output?

```
int a=10, b=3;
printf("%f",(float)a/b);
```

A) 3.000000
B) 3.333333
C) 3
D) Error

■ Correct Answer: B) 3.333333

Explanation: Casting a to float before division makes it floating-point: 10.0/3 ≈ 3.333333.

Q30. What is (int)3.7 + (int)(-3.7) in C?

A) 0
B) -1
C) 1
D) -7

■ Correct Answer: A) 0

Explanation: (int)3.7=3, (int)(-3.7)=-3; 3+(-3)=0.

Q31. Which header file is required for pow() and sqrt() functions?

A) stdlib.h
B) string.h
C) math.h
D) stdio.h

■ Correct Answer: C) math.h

Explanation: Mathematical functions like pow() and sqrt() are declared in <math.h>.

Q32. What is the output?

```
int a=5;
printf("%d",a*=2+3);
```

A) 10
B) 15
C) 25
D) 13

■ Correct Answer: C) 25

Explanation: a*=2+3 → a = a*(2+3) = 5*5 = 25 (right side evaluated first).

Q33. What is the result of 1+2+3*4*5?

A) 300
B) 63
C) 60
D) 65

■ Correct Answer: B) 63
Explanation: Precedence: 3*4=12, 12*5=60 → 1+2+60=63.

Q34. Integer division in C truncates towards:

A) Positive infinity
B) Zero
C) Negative infinity
D) Nearest integer

■ Correct Answer: B) Zero
Explanation: C99 mandates truncation towards zero for integer division.

Q35. What is the output?

```
printf("%d", -10 % 3);
```

A) -1
B) 1
C) 2
D) -2

■ Correct Answer: A) -1
Explanation: C99/C11: -10%3 = -1 (sign follows dividend).

Q36. How many arithmetic operators does C have?

A) 4
B) 5
C) 6
D) 3

■ Correct Answer: B) 5
Explanation: C has 5 arithmetic operators: +, -, *, /, %.

Q37. What is the output?

```
int x=10,y=4;
printf("%d",x-y*2+x/y);
```

A) 5
B) 4
C) 6
D) 2

■ Correct Answer: B) 4

Explanation: y*2=8, x/y=2 → 10-8+2=4.

Q38. Which of the following will give a float result in C?

A) 5/2
B) 5.0/2
C) 5/2.0f
D) Both B and C

■ Correct Answer: D) Both B and C

Explanation: If at least one operand is floating-point, the result is floating-point.

Q39. What is the value of 11 % 4 + 11 / 4?

A) 4
B) 5
C) 6
D) 3

■ Correct Answer: B) 5

Explanation: 11%4=3, 11/4=2 → 3+2=5.

Q40. Unary minus (-) has _____ precedence than binary subtraction (-)?

A) Lower
B) Higher
C) Equal
D) Undefined

■ Correct Answer: B) Higher

Explanation: Unary minus has higher precedence than binary minus/addition.

Section 2: Relational Operators

Q41. Which relational operator checks for equality in C?

A) =
B) ==
C) !=
D) ===

> ■ **Correct Answer: B) ==**
> **Explanation:** == is the equality operator; = is assignment.

Q42. What is the output?

```
printf("%d", 5 > 3);
```

A) 1
B) 0
C) True
D) Error

> ■ **Correct Answer: A) 1**
> **Explanation:** Relational expressions return 1 (true) or 0 (false) in C.

Q43. What is the output?

```
printf("%d", 5 == 5.0);
```

A) 0
B) 1
C) Error
D) Undefined

> ■ **Correct Answer: B) 1**
> **Explanation:** 5 is promoted to double (5.0); 5.0 == 5.0 is true (1).

Q44. Which symbol represents 'not equal to' in C?

A) <>
B) !=
C) #
D) ~=

> ■ **Correct Answer: B) !=**
> **Explanation:** != is the not-equal-to operator in C.

Q45. What is the output?

```
int a=3, b=5;
printf("%d", a>=b);
```

A) 0
B) 1
C) -1
D) Error

■ Correct Answer: A) 0
Explanation: 3 >= 5 is false; result is 0.

Q46. What is the output?

```
printf("%d", 0 == 0.0);
```

A) 0
B) 1
C) Undefined
D) Error

■ Correct Answer: B) 1
Explanation: 0 promoted to 0.0; 0.0 == 0.0 is true (1).

Q47. What is the result of (10 != 10)?

A) 1
B) 0
C) -1
D) True

■ Correct Answer: B) 0
Explanation: 10 != 10 is false; result is 0.

Q48. What is the output?

```
int x=5;
printf("%d", x>3 && x<10);
```

A) 0
B) 2
C) 1
D) Error

■ Correct Answer: C) 1
Explanation: Both conditions true; logical AND gives 1.

Q49. Which of the following is TRUE about relational operators in C?

A) They return -1, 0, or 1
B) They return only 0 or 1
C) They work on strings directly
D) They are right-associative

> ■ **Correct Answer: B) They return only 0 or 1**
> **Explanation:** Relational operators return integer 0 (false) or 1 (true).

Q50. What is the output?

```
printf("%d", 'A' < 'B');
```

A) 0
B) 1
C) Error
D) -1

> ■ **Correct Answer: B) 1**
> **Explanation:** ASCII 'A'=65, 'B'=66; 65 < 66 is true (1).

Q51. What is the output?

```
int a=5,b=10;
printf("%d", a<b<3);
```

A) 1
B) 0
C) 3
D) Error

> ■ **Correct Answer: A) 1**
> **Explanation:** Left-to-right: a<b = 1 (true); then 1<3 = 1 (true).

Q52. What is the precedence order (highest to lowest)?

A) `==, !=, <, >`
B) `<, >, <=, >=, ==, !=`
C) `!=, ==, >=, <=`
D) All same

> ■ **Correct Answer: B) <, >, <=, >=, ==, !=**
> **Explanation:** <, >, <=, >= have higher precedence than == and !=.

Q53. What is the output?

```
printf("%d", 3<=3);
```

A) 0
B) 1
C) Error
D) 3

■ Correct Answer: B) 1
Explanation: 3 <= 3 is true; returns 1.

Q54. Comparing a pointer and an integer directly in C is:

A) Valid
B) Undefined behaviour
C) Always 0
D) Always 1

■ Correct Answer: B) Undefined behaviour
Explanation: Comparing a pointer with an integer (other than 0/NULL) is undefined behaviour.

Q55. What is the output?

```
char c='z';
printf("%d", c>'a');
```

A) 0
B) 1
C) 25
D) Error

■ Correct Answer: B) 1
Explanation: 'z'=122, 'a'=97; 122>97 is true (1).

Q56. What is the output?

```
printf("%d", 1 == 1 == 1);
```

A) 1
B) 0
C) 2
D) Error

■ Correct Answer: A) 1
Explanation: Left-to-right: 1==1=1; then 1==1=1.

Q57. What is the output?

```
printf("%d", 5 != 4);
```

A) 0
B) 1
C) 5
D) 4

■ Correct Answer: B) 1
Explanation: 5 != 4 is true; returns 1.

Q58. int a=0; What is the value of (a==0)?

A) 0
B) 1
C) -1
D) Undefined

■ Correct Answer: B) 1
Explanation: a is 0, so a==0 is true; returns 1.

Q59. What is the output?

```
printf("%d", (2>1)+(3>2));
```

A) 2
B) 1
C) 0
D) 3

■ Correct Answer: A) 2
Explanation: 2>1=1, 3>2=1; 1+1=2.

Q60. Which relational operator returns 1 only when left operand is strictly less?

A) `<=`
B) `<`
C) `!=`
D) `==`

■ Correct Answer: B) <
Explanation: < (less than) is true only when left < right (strict).

Q61. What is the output?

```
int x=5;
printf("%d",!(x==5));
```

A) 1
B) 0
C) -1
D) Error

■ Correct Answer: B) 0

Explanation: x==5 is 1 (true); !1 = 0.

Q62. Relational operators in C have _____ associativity.

A) Right to left
B) Left to right
C) Non-associative
D) None

■ Correct Answer: B) Left to right

Explanation: Relational operators associate left to right.

Q63. What is the output?

```
printf("%d", 4>=4);
```

A) 0
B) 1
C) 4
D) Error

■ Correct Answer: B) 1

Explanation: 4 >= 4 is true; returns 1.

Q64. Which of the following correctly tests if x is between 1 and 10 (inclusive)?

A) `1 <= x <= 10`
B) `x >= 1 && x <= 10`
C) x between 1 and 10
D) `1 =< x =< 10`

■ Correct Answer: B) x >= 1 && x <= 10

Explanation: Use logical AND to combine two relational checks.

Q65. What is the output?

```
int a=10,b=20;
printf("%d %d",a>b, a<b);
```

A) 0 1
B) 1 0
C) 1 1
D) 0 0

> ■ **Correct Answer: A) 0 1**
> **Explanation:** 10>20 is false (0); 10<20 is true (1).

Q66. What is the output?

```
printf("%d", 'a'=='a');
```

A) 0
B) 1
C) 97
D) Error

> ■ **Correct Answer: B) 1**
> **Explanation:** Character comparison by ASCII value; 97==97 is true (1).

Q67. Which operator has lower precedence: > or ==?

A) >
B) ==
C) Both equal
D) Depends

> ■ **Correct Answer: B) ==**
> **Explanation:** == and != have lower precedence than <, >, <=, >=.

Q68. What is the output?

```
int a=5;
printf("%d", a==(a=10));
```

A) 0
B) 1
C) Undefined
D) Error

> ■ **Correct Answer: C) Undefined**
> **Explanation:** Modifying and reading a in the same expression without sequence point is UB.

Q69. What is the output?

```
printf("%d", -1 < 1u); // 1u is unsigned
```

A) 0
B) 1
C) Undefined
D) Error

■ Correct Answer: A) 0

Explanation: -1 as signed is converted to a large unsigned value; it's NOT less than 1u.

Q70. What is the output?

```
int x=3,y=3;
printf("%d",x>=y);
```

A) 0
B) 1
C) 3
D) Error

■ Correct Answer: B) 1

Explanation: 3 >= 3 is true; returns 1.

Q71. In C, using = instead of == in a condition will:

A) Cause compile error
B) Always be false
C) Assign and use the assigned value as condition
D) Be same as ==

■ Correct Answer: C) Assign and use the assigned value as condition

Explanation: = assigns and returns the value; non-zero is treated as true.

Q72. What is the output?

```
if(5>3)
printf("Yes");
```

else

```
printf("No");
```

A) No
B) Yes
C) Error
D) Nothing

■ Correct Answer: B) Yes

Explanation: 5>3 is true so 'Yes' is printed.

Q73. What is the output?

```
printf("%d", (3>2)-(2>3));
```

A) 0
B) 1
C) -1
D) 2

■ Correct Answer: B) 1
Explanation: 3>2=1, 2>3=0; 1-0=1.

Q74. Relational operators are used primarily in:

A) Mathematical expressions
B) Decision-making and loop control
C) String operations
D) Bitwise operations

■ Correct Answer: B) Decision-making and loop control
Explanation: Relational operators are used in if, while, for etc. for decision control.

Q75. What is the output?

```
int a=5,b=5;
printf("%d",a!=b);
```

A) 0
B) 1
C) 5
D) Error

■ Correct Answer: A) 0
Explanation: 5 != 5 is false; returns 0.

Q76. How many relational operators are in C?

A) 4
B) 6
C) 5
D) 3

■ Correct Answer: B) 6
Explanation: C has 6 relational operators: <, >, <=, >=, ==, !=.

Q77. What is the output?

```
printf("%d", 0 != 1);
```

A) 0
B) 1
C) -1
D) Error

> ■ **Correct Answer: B) 1**
> **Explanation:** 0 != 1 is true; returns 1.

Q78. What is the output?

```
int x=10;
printf("%d", x>5>3);
```

A) 1
B) 0
C) 3
D) Error

> ■ **Correct Answer: B) 0**
> **Explanation:** Left-to-right: x>5=1; then 1>3=0.

Q79. The expression (a > b) in C evaluates to type:

A) bool
B) int
C) float
D) char

> ■ **Correct Answer: B) int**
> **Explanation:** In C, relational expressions evaluate to int (0 or 1).

Q80. What is the output?

```
printf("%d", 10 == 10.0);
```

A) 0
B) 1
C) Error
D) Undefined

> ■ **Correct Answer: B) 1**
> **Explanation:** 10 promoted to 10.0; 10.0==10.0 is true (1).

Section 3: Logical Operators

Q81. Which symbol represents logical AND in C?

A) &
B) &&
C) AND
D) and

> ■ **Correct Answer: B) &&**
> **Explanation:** && is the logical AND; & is bitwise AND.

Q82. What is the output?

```
printf("%d", 1 && 0);
```

A) 0
B) 1
C) -1
D) Error

> ■ **Correct Answer: A) 0**
> **Explanation:** Logical AND is 0 if either operand is 0.

Q83. What is the output?

```
printf("%d", 0 || 1);
```

A) 0
B) 1
C) -1
D) Error

> ■ **Correct Answer: B) 1**
> **Explanation:** Logical OR is 1 if at least one operand is non-zero.

Q84. What is the output?

```
printf("%d", !0);
```

A) 0
B) 1
C) -1
D) Error

> ■ **Correct Answer: B) 1**
> **Explanation:** Logical NOT of 0 is 1.

Q85. What is the output?

```
printf("%d", !5);
```

A) 0
B) 1
C) -5
D) Error

■ Correct Answer: A) 0

Explanation: !5: any non-zero is true; NOT true = 0.

Q86. Short-circuit evaluation: in (a && b), if a is 0, what happens to b?

A) b is evaluated
B) b is not evaluated
C) b is set to 0
D) Error

■ Correct Answer: B) b is not evaluated

Explanation: && short-circuits; if left is false, right is not evaluated.

Q87. Short-circuit evaluation: in (a || b), if a is non-zero, what happens to b?

A) b is evaluated
B) b is not evaluated
C) b is set to 1
D) Error

■ Correct Answer: B) b is not evaluated

Explanation: || short-circuits; if left is true, right is not evaluated.

Q88. What is the output?

```
int a=0,b=5;
printf("%d", a&&(b=10));
printf(" %d",b);
```

A) 0 10
B) 0 5
C) 1 10
D) 1 5

■ Correct Answer: B) 0 5

Explanation: a is 0; && short-circuits; b=10 never executes; b stays 5.

Q89. What is the output?

```
printf("%d", 5>3 && 2<4);
```

A) 0
B) 1
C) 2
D) Error

> ■ **Correct Answer: B) 1**
> **Explanation:** 5>3=1, 2<4=1; 1&&1=1.

Q90. What is the output?

```
printf("%d", 5>3 || 2>4);
```

A) 0
B) 1
C) 2
D) Error

> ■ **Correct Answer: B) 1**
> **Explanation:** 5>3=1 (true); || short-circuits; result=1.

Q91. What is the output?

```
printf("%d", 0 || 0);
```

A) 0
B) 1
C) -1
D) Error

> ■ **Correct Answer: A) 0**
> **Explanation:** Both operands 0; logical OR = 0.

Q92. What is the output?

```
printf("%d", !(5==5));
```

A) 0
B) 1
C) -1
D) Error

> ■ **Correct Answer: A) 0**
> **Explanation:** 5==5=1; !1=0.

Q93. Which has higher precedence: && or ||?

A) ||
B) &&
C) Equal
D) Depends on operands

■ Correct Answer: B) &&
Explanation: && has higher precedence than ||.

Q94. What is the output?

```
printf("%d", 0 && 0 || 1);
```

A) 0
B) 1
C) -1
D) Error

■ Correct Answer: B) 1
Explanation: && first: 0&&0=0; then 0||1=1.

Q95. What is the output?

```
printf("%d", 1 || 0 && 0);
```

A) 0
B) 1
C) Error
D) Undefined

■ Correct Answer: B) 1
Explanation: && before ||: 0&&0=0; then 1||0=1.

Q96. Logical NOT (!) is a _____ operator.

A) Binary
B) Ternary
C) Unary
D) Bitwise

■ Correct Answer: C) Unary
Explanation: ! operates on a single operand (unary).

Q97. What is the output?

```
int x=5,y=0;
printf("%d", x || y);
```

A) 0
B) 1
C) 5
D) Error

■ Correct Answer: B) 1

Explanation: x=5 is non-zero (true); || gives 1.

Q98. What is the output?

```
printf("%d",!!5);
```

A) 5
B) 1
C) 0
D) -5

■ Correct Answer: B) 1

Explanation: !5=0; !0=1.

Q99. What is the truth table result of 1 && 1?

A) 0
B) 1
C) 2
D) Undefined

■ Correct Answer: B) 1

Explanation: 1 && 1 = 1 (both true).

Q100. What is the output?

```
int a=1;
printf("%d",a++||a++);
```

A) 1
B) 2
C) Undefined
D) 0

■ Correct Answer: A) 1

Explanation: a++=1 (true); || short-circuits; result is 1. (a may be 2 or 3 — UB order of side effects, but result of || is 1.)

Q101. What is the output?

```
printf("%d", 0 || 0 || 1);
```

A) 0
B) 1
C) 3
D) Error

■ Correct Answer: B) 1

Explanation: Left-to-right: 0||0=0; 0||1=1.

Q102. In C, which value is considered FALSE in a logical context?

A) Any negative number
B) 0
C) NULL only
D) 0 and NULL

■ Correct Answer: B) 0

Explanation: Only 0 is false; any non-zero (including negative) is true.

Q103. What is the output?

```
printf("%d", -5 && 0);
```

A) 0
B) -5
C) 1
D) Error

■ Correct Answer: A) 0

Explanation: -5 is non-zero (true); 0 is false; true&&false=0.

Q104. Which logical operator can be used to negate a condition?

A) ~
B) !
C) NOT
D) ^

■ Correct Answer: B) !

Explanation: ! is the logical NOT operator.

Q105. What is the output?

```
int x=0;
if(!x)
printf("Zero");
```

A) Zero
B) Nothing
C) Error
D) 1

✅ Correct Answer: A) Zero
Explanation: !0=1 (true); 'Zero' is printed.

Q106. What is the output?

```
printf("%d", (3>2)&&(4>3));
```

A) 0
B) 1
C) 2
D) Error

✅ Correct Answer: B) 1
Explanation: Both true; 1&&1=1.

Q107. Logical operators && and || return values of type:

A) bool
B) int (0 or 1)
C) char
D) float

✅ Correct Answer: B) int (0 or 1)
Explanation: In C, logical operators return int 0 or 1.

Q108. What is the output?

```
printf("%d", 5<3 || 3>2);
```

A) 0
B) 1
C) Error
D) 2

✅ Correct Answer: B) 1
Explanation: 5<3=0, 3>2=1; 0||1=1.

Q109. What is the output?

```
printf("%d", 5 && 3);
```

A) 3
B) 5
C) 1
D) 0

■ Correct Answer: C) 1
Explanation: Both non-zero; logical AND returns 1 (not the actual values).

Q110. Short-circuit in && means:

A) Right operand evaluated first
B) Left operand skipped if right is 0
C) Right operand NOT evaluated if left is 0
D) Both always evaluated

■ Correct Answer: C) Right operand NOT evaluated if left is 0
Explanation: In a&&b, if a is 0 (false), b is not evaluated.

Q111. What is the output?

```
printf("%d",!(0==0));
```

A) 0
B) 1
C) Error
D) -1

■ Correct Answer: A) 0
Explanation: 0==0=1; !1=0.

Q112. What is the output?

```
int a=1,b=2;
printf("%d", a<b || b<a);
```

A) 0
B) 1
C) 2
D) Error

■ Correct Answer: B) 1
Explanation: a<b=1 (true); || short-circuits; result=1.

Q113. What is the precedence order (high to low): &&, ||, !?

A) &&, ||, !
B) !, &&, ||
C) ||, &&, !
D) All equal

> ■ **Correct Answer: B) !, &&, ||**
> **Explanation:** ! (NOT) has highest, then && (AND), then || (OR).

Q114. What is the output?

```
printf("%d", 2 || 3);
```

A) 1
B) 2
C) 3
D) 6

> ■ **Correct Answer: A) 1**
> **Explanation:** Both non-zero; logical OR returns 1.

Q115. What is the output?

```
printf("%d", 1 && 2 && 3);
```

A) 3
B) 6
C) 1
D) 0

> ■ **Correct Answer: C) 1**
> **Explanation:** All non-zero; logical AND returns 1.

Q116. Logical operators vs bitwise operators: which evaluates both operands always?

A) Logical
B) Bitwise
C) Both
D) Neither

> ■ **Correct Answer: B) Bitwise**
> **Explanation:** Bitwise operators (&, |, ^) always evaluate both operands; logical operators short-circuit.

Q117. What is the output?

```
printf("%d", 0 && printf("Hi"));
```

A) Hi0
B) 0
C) Hi
D) Error

■ Correct Answer: B) 0

Explanation: && short-circuits; printf("Hi") never called; result is 0.

Q118. What is the output?

```
printf("%d", 1 || printf("Hi"));
```

A) 1
B) Hi1
C) Hi
D) Error

■ Correct Answer: A) 1

Explanation: || short-circuits when left is 1; printf not called; result is 1.

Q119. What is the output?

```
int a=5;
printf("%d", a>0 && a<10);
```

A) 0
B) 1
C) 5
D) Error

■ Correct Answer: B) 1

Explanation: Both conditions true; 1&&1=1.

Q120. What is the output?

```
printf("%d", 0 || 0 || 0);
```

A) 0
B) 1
C) 3
D) Error

■ Correct Answer: A) 0

Explanation: All zeros; OR returns 0.

Section 4: Assignment Operators

Q121. What is the output?

```
int a=5;
a+=3;
printf("%d",a);
```

A) 3
B) 8
C) 15
D) 5

> ■ **Correct Answer: B) 8**
> **Explanation:** a+=3 means a=a+3=5+3=8.

Q122. What does the operator -= do?

A) Logical subtraction
B) `a = a - b`
C) Decrement by 1
D) Bitwise subtract

> ■ **Correct Answer: B) a = a - b**
> **Explanation:** -= is compound assignment: a-=b means a=a-b.

Q123. What is the output?

```
int x=10;
x-=4;
printf("%d",x);
```

A) 14
B) 6
C) 40
D) 4

> ■ **Correct Answer: B) 6**
> **Explanation:** x=x-4=10-4=6.

Q124. What is the output?

```
int a=3;
a*=4;
printf("%d",a);
```

A) 7
B) 43
C) 12
D) 1

> ■ **Correct Answer: C) 12**
> **Explanation:** a=a*4=3*4=12.

Q125. What is the output?

```
int a=20;
a/=4;
printf("%d",a);
```

A) 16
B) 5
C) 24
D) 80

■ Correct Answer: B) 5
Explanation: a=a/4=20/4=5.

Q126. What is the output?

```
int a=17;
a%=5;
printf("%d",a);
```

A) 3
B) 2
C) 5
D) 1

■ Correct Answer: B) 2
Explanation: a=a%5=17%5=2.

Q127. The statement a=b=c=5; in C means:

A) Error
B) a, b, c all become 5 (right-to-left)
C) Only a becomes 5
D) Only c becomes 5

■ Correct Answer: B) a, b, c all become 5 (right-to-left)
Explanation: Assignment is right-associative; c=5, b=5, a=5.

Q128. What is the output?

```
int a,b,c;
a=b=c=10;
printf("%d %d %d",a,b,c);
```

A) 10 10 10
B) 0 0 10
C) 10 0 0
D) Error

■ Correct Answer: A) 10 10 10
Explanation: Chained assignment assigns 10 to all.

Q129. What is the associativity of the assignment operator?

A) Left to right
B) Right to left
C) Non-associative
D) None

✅ Correct Answer: B) Right to left
Explanation: Assignment operator is right-associative.

Q130. What is the output?

```
int a=5;
a+=a;
printf("%d",a);
```

A) 5
B) 0
C) 10
D) 25

✅ Correct Answer: C) 10
Explanation: a=a+a=5+5=10.

Q131. What is the output?

```
int x=10;
x*=x-5;
printf("%d",x);
```

A) 50
B) 25
C) 100
D) 5

✅ Correct Answer: A) 50
Explanation: Right side evaluated first: x-5=5; x=x*5=10*5=50.

Q132. Which operator is used for compound bitwise AND assignment?

A) `&&=`
B) `&=`
C) `|=`
D) `^=`

✅ Correct Answer: B) &=
Explanation: &= is the compound bitwise AND assignment operator.

Q133. What is the output?

```
int a=12;
a&=10;
printf("%d",a);
```

A) 10
B) 8
C) 2
D) 14

■ Correct Answer: B) 8

Explanation: 12=1100, 10=1010; 1100&1010=1000=8.

Q134. What is the output?

```
int a=5;
a|=3;
printf("%d",a);
```

A) 5
B) 3
C) 7
D) 1

■ Correct Answer: C) 7

Explanation: 5=0101, 3=0011; 0101|0011=0111=7.

Q135. What is the output?

```
int a=15;
a^=9;
printf("%d",a);
```

A) 6
B) 7
C) 5
D) 12

■ Correct Answer: A) 6

Explanation: 15=1111, 9=1001; 1111^1001=0110=6.

Q136. What is the output?

```
int a=8;
a>>=1;
printf("%d",a);
```

A) 16
B) 4
C) 8
D) 2

■ Correct Answer: B) 4

Explanation: a=8=1000; >>1 shifts right: 0100=4.

Q137. What is the output?

```
int a=3;
a<<=2;
printf("%d",a);
```

A) 6
B) 12
C) 1
D) 9

■ Correct Answer: B) 12

Explanation: a=3=011; <<2 shifts left: 1100=12.

Q138. What does a+=b mean in C?

```
A) a = a + a + b
B) a = a + b
C) a = b + 1
D) a = a - b
```

■ Correct Answer: B) a = a + b

Explanation: += adds right operand to left and stores result in left.

Q139. What is the output?

```
int a=10,b=5;
a/=b;
printf("%d",a);
```

A) 2
B) 50
C) 5
D) 15

■ Correct Answer: A) 2

Explanation: a=a/b=10/5=2.

Q140. Which of the following is NOT a valid compound assignment operator?

A) +=
B) -=
C) ==
D) %=

> ■ **Correct Answer: C) ==**
> **Explanation:** == is equality comparison, not assignment.

Q141. What is the output?

```
int a=10;
a%=3;
printf("%d",a);
```

A) 1
B) 2
C) 0
D) 3

> ■ **Correct Answer: A) 1**
> **Explanation:** a=10%3=1.

Q142. What is the output?

```
int x=4;
x+=x*=2;
printf("%d",x);
```

A) 12
B) 16
C) 8
D) Undefined

> ■ **Correct Answer: D) Undefined**
> **Explanation:** Modifying x twice without sequence point is undefined behaviour.

Q143. Assignment operator (=) returns:

A) Nothing
B) The left operand value
C) The assigned value
D) 1

> ■ **Correct Answer: C) The assigned value**
> **Explanation:** The assignment expression evaluates to the value being assigned.

Q144. What is the output?

```
int a;
printf("%d",a=5);
```

A) 5
B) 0
C) Error
D) Undefined

> ■ **Correct Answer: A) 5**
> **Explanation:** a=5 assigns 5 and the expression evaluates to 5.

Q145. How many compound assignment operators does C have?

A) 5
B) 8
C) 10
D) 12

> ■ **Correct Answer: C) 10**
> **Explanation:** C has 10 compound assignment operators: +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=.

Q146. What is the output?

```
int a=6,b=4;
b=a=10;
printf("%d %d",a,b);
```

A) 6 10
B) 10 10
C) 10 6
D) Error

> ■ **Correct Answer: B) 10 10**
> **Explanation:** Right-to-left: a=10, then b=a=10.

Q147. What is the output?

```
int a=5;
if(a=0)
printf("T");
else
printf("F");
```

A) T
B) F
C) Error
D) Nothing

> ■ **Correct Answer: B) F**
> **Explanation:** a=0 assigns 0; condition is 0 (false); else branch executes.

Q148. Which of the following correctly swaps a and b WITHOUT a temp variable using +=/-=?

```
A) a+=b; b-=a; a+=b;
B) a-=b; b+=a; a=b-a;
C) a+=b; b=a-b; a-=b;
D) b+=a; a-=b; b-=a;
```

■ Correct Answer: C) a+=b; b=a-b; a-=b;

Explanation: a+=b makes a=a+b; b=a-b gives original a; a-=b gives original b.

Q149. What is the output?

```
int x=100;
x/=10;
x/=2;
printf("%d",x);
```

A) 5
B) 10
C) 50
D) 2

■ Correct Answer: A) 5

Explanation: 100/10=10; 10/2=5.

Q150. What is the output?

```
int a=7;
a^=a;
printf("%d",a);
```

A) 7
B) 0
C) 14
D) 1

■ Correct Answer: B) 0

Explanation: XOR of any number with itself is 0; classic swap trick.

Q151. What is the output?

```
int a=2;
a<<=3;
printf("%d",a);
```

A) 6
B) 16
C) 5
D) 8

■ Correct Answer: B) 16

Explanation: 2=010; <<3: 10000=16.

Q152. What is the output?

```
int a=16;
a>>=2;
printf("%d",a);
```

A) 4
B) 8
C) 2
D) 64

■ Correct Answer: A) 4
Explanation: 16=10000; >>2: 00100=4.

Q153. In C, the expression (a=b) inside if means:

A) Compare a and b
B) Assign b to a, test if a is non-zero
C) Always true
D) Always false

■ Correct Answer: B) Assign b to a, test if a is non-zero
Explanation: = assigns; the resulting value of a is used as condition.

Q154. What is the output?

```
int a=5,b=3;
a-=b+=2;
printf("%d %d",a,b);
```

A) 2 5
B) 0 5
C) 5 5
D) Error

■ Correct Answer: B) 0 5
Explanation: Right-to-left: b+=2 → b=5; a-=b (now 5) → a=5-5=0.

Q155. What is the output?

```
int a=10;
a+=5; a-=2; a*=3;
printf("%d",a);
```

A) 39
B) 36
C) 40
D) 30

■ Correct Answer: A) 39
Explanation: a=10+5=15; 15-2=13; 13*3=39.

Section 5: Bitwise Operators

Q156. What is the output?

```
printf("%d", 5 & 3);
```

A) 7
B) 1
C) 5
D) 3

■ Correct Answer: B) 1
Explanation: 5=0101, 3=0011; AND: 0001=1.

Q157. What is the output?

```
printf("%d", 5 | 3);
```

A) 7
B) 1
C) 8
D) 6

■ Correct Answer: A) 7
Explanation: 5=0101, 3=0011; OR: 0111=7.

Q158. What is the output?

```
printf("%d", 5 ^ 3);
```

A) 7
B) 6
C) 1
D) 8

■ Correct Answer: B) 6
Explanation: 5=0101, 3=0011; XOR: 0110=6.

Q159. What is the output?

```
printf("%d", ~5);
```

A) -6
B) -5
C) 5
D) 0

■ Correct Answer: A) -6
Explanation: ~5 inverts all bits; in two's complement: ~5 = -(5+1) = -6.

Q160. What is the output?

```
printf("%d", 1 << 3);
```

A) 3
B) 8
C) 4
D) 6

> ■ **Correct Answer: B) 8**
> **Explanation:** 1 shifted left 3 = 2^3 = 8.

Q161. What is the output?

```
printf("%d", 16 >> 2);
```

A) 4
B) 8
C) 2
D) 64

> ■ **Correct Answer: A) 4**
> **Explanation:** 16=10000; >>2 = 00100 = 4.

Q162. Which bitwise operator is used to check if a bit is set?

A) |
B) &
C) ^
D) ~

> ■ **Correct Answer: B) &**
> **Explanation:** AND with a mask: if (x & mask) checks if bits in mask are set in x.

Q163. How to set bit 3 (0-indexed from right) of variable x?

```
A) x = x & (1<<3)
B) x = x | (1<<3)
C) x = x ^ (1<<3)
D) x = x - (1<<3)
```

> ■ **Correct Answer: B) x = x | (1<<3)**
> **Explanation:** OR with the bit mask sets the bit.

Q164. How to clear bit 2 of variable x?

```
A) x = x & ~(1<<2)
B) x = x | (1<<2)
C) x = x ^ (1<<2)
D) x = x >> 2
```

> ■ **Correct Answer: A) x = x & ~(1<<2)**
> **Explanation:** AND with the complement of the bit mask clears the bit.

Q165. How to toggle bit 4 of variable x?

```
A) x = x | (1<<4)
B) x = x & ~(1<<4)
C) x = x ^ (1<<4)
D) x = x >> 4
```

> ■ **Correct Answer: C) x = x ^ (1<<4)**
> **Explanation:** XOR with the bit mask toggles the bit.

Q166. What is the output?

```
printf("%d", 12 & 10);
```

A) 8
B) 14
C) 2
D) 6

> ■ **Correct Answer: A) 8**
> **Explanation:** 12=1100, 10=1010; AND: 1000=8.

Q167. What is the output?

```
printf("%d", 12 | 10);
```

A) 8
B) 14
C) 2
D) 22

> ■ **Correct Answer: B) 14**
> **Explanation:** 12=1100, 10=1010; OR: 1110=14.

Q168. What is the output?

```
printf("%d", 12 ^ 10);
```

A) 8
B) 6
C) 2
D) 14

■ Correct Answer: B) 6
Explanation: 12=1100, 10=1010; XOR: 0110=6.

Q169. What is the value of ~0 on a 32-bit system?

A) 0
B) 1
C) -1
D) INT_MAX

■ Correct Answer: C) -1
Explanation: ~0 flips all bits to 1; in two's complement this is -1.

Q170. What is the output?

```
printf("%d", 1 << 4);
```

A) 4
B) 8
C) 16
D) 5

■ Correct Answer: C) 16
Explanation: 1 << 4 = 2^4 = 16.

Q171. What is the output?

```
printf("%d", 32 >> 3);
```

A) 4
B) 8
C) 128
D) 2

■ Correct Answer: A) 4
Explanation: 32 >> 3 = 32/2^3 = 32/8 = 4.

Q172. Which operator gives 0 when applied to equal operands?

A) &
B) |
C) ^
D) ~

> ■ **Correct Answer: C) ^**
> **Explanation:** XOR of equal values is always 0 (used for detecting equality or clearing).

Q173. What is the output?

```
int x=5;
printf("%d",x&1);
```

A) 0
B) 1
C) 5
D) 4

> ■ **Correct Answer: B) 1**
> **Explanation:** 5=0101; 5&1=0001=1 (odd number has bit 0 set).

Q174. What is the output?

```
int x=4;
printf("%d",x&1);
```

A) 0
B) 1
C) 4
D) 5

> ■ **Correct Answer: A) 0**
> **Explanation:** 4=0100; 4&1=0000=0 (even number has bit 0 clear).

Q175. Multiplying by 2 using bit shift:

A) x >> 1
B) x << 1
C) x & 2
D) x | 2

> ■ **Correct Answer: B) x << 1**
> **Explanation:** Left shift by 1 is equivalent to multiplying by 2.

Q176. Dividing by 4 using bit shift:

A) x << 2
B) x >> 2
C) x & 4
D) x | 4

> ■ **Correct Answer: B) x >> 2**
> **Explanation:** Right shift by 2 is equivalent to dividing by 4 (for non-negative).

Q177. What is the output?

```
printf("%d", 7 & 7);
```

A) 0
B) 7
C) 14
D) 1

> ■ **Correct Answer: B) 7**
> **Explanation:** 7 & 7 = 7 (AND with itself).

Q178. What is the output?

```
printf("%d", 7 ^ 7);
```

A) 7
B) 14
C) 0
D) 1

> ■ **Correct Answer: C) 0**
> **Explanation:** XOR of any number with itself = 0.

Q179. What is the output?

```
printf("%d", 7 | 0);
```

A) 0
B) 7
C) 1
D) 8

> ■ **Correct Answer: B) 7**
> **Explanation:** OR with 0 leaves value unchanged.

Q180. What is the output?

```
printf("%d", 7 & 0);
```

A) 0
B) 7
C) 1
D) 8

■ Correct Answer: A) 0
Explanation: AND with 0 gives 0.

Q181. To extract bits 3-5 (0-indexed) of x, the mask is:

A) 0x07
B) 0x38
C) 0x1F
D) 0x0F

■ Correct Answer: B) 0x38
Explanation: Bits 3,4,5 = 0b00111000 = 0x38.

Q182. What is ~(~x) equal to?

A) 0
B) -1
C) x
D) ~x

■ Correct Answer: C) x
Explanation: Double NOT restores original value.

Q183. What is the output?

```
printf("%d", 0xFF & 0x0F);
```

A) 255
B) 15
C) 240
D) 0

■ Correct Answer: B) 15
Explanation: 0xFF=11111111, 0x0F=00001111; AND=00001111=15.

Q184. What is the output?

```
printf("%d", 1 << 0);
```

A) 0
B) 1
C) 2
D) Error

■ Correct Answer: B) 1
Explanation: 1 shifted left 0 positions = 1.

Q185. What is the output?

```
printf("%d", 15 >> 1);
```

A) 30
B) 7
C) 8
D) 3

■ Correct Answer: B) 7
Explanation: 15=1111; >>1 = 0111 = 7.

Q186. Bitwise OR is used to:

A) Clear bits
B) Set bits
C) Toggle bits
D) Count bits

■ Correct Answer: B) Set bits
Explanation: ORing with 1 sets bits; with 0 leaves unchanged.

Q187. Bitwise AND is used to:

A) Set bits
B) Toggle bits
C) Clear/mask bits
D) Shift bits

■ Correct Answer: C) Clear/mask bits
Explanation: ANDing with 0 clears bits; with 1 preserves them.

Q188. What is the output?

```
printf("%d", 5 << 1 | 1);
```

A) 10
B) 11
C) 9
D) 6

■ Correct Answer: B) 11

Explanation: Precedence: << before |: 5<<1=10; 10|1=0b1011=11.

Q189. What is the output?

```
printf("%d", 0xA & 0x5);
```

A) 15
B) 0
C) 5
D) 10

■ Correct Answer: B) 0

Explanation: 0xA=1010, 0x5=0101; AND=0000=0.

Q190. What is the output?

```
printf("%d", 0xA | 0x5);
```

A) 15
B) 5
C) 10
D) 0

■ Correct Answer: A) 15

Explanation: 0xA=1010, 0x5=0101; OR=1111=15.

Q191. Right shift of a negative number is implementation-defined because:

A) Negative numbers can't be shifted
B) Sign bit handling varies (arithmetic vs logical shift)
C) It always produces 0
D) It always produces -1

■ Correct Answer: B) Sign bit handling varies (arithmetic vs logical shift)

Explanation: For signed negatives, right shift behaviour is implementation-defined in C.

Q192. What is the output?

```
printf("%d", 8>>1<<1);
```

A) 8
B) 4
C) 16
D) 2

■ **Correct Answer: A) 8**
Explanation: Left-to-right: 8>>1=4; 4<<1=8.

Q193. What is the output?

```
printf("%d", ~(-1));
```

A) -2
B) 0
C) 1
D) -1

■ **Correct Answer: B) 0**
Explanation: ~(-1): -1 is all 1s in binary; ~(all 1s) = all 0s = 0.

Q194. What is the maximum number of bits that can be shifted in a 32-bit int?

A) 31
B) 32
C) 33
D) Unlimited

■ **Correct Answer: A) 31**
Explanation: Shifting by >= width of type is undefined behaviour; max defined shift is 31.

Q195. What is the output?

```
printf("%d", (3^5)&7);
```

A) 6
B) 7
C) 5
D) 3

■ **Correct Answer: A) 6**
Explanation: 3^5: 011^101=110=6; 6&7: 110&111=110=6.

Section 6: Increment / Decrement Operators

Q196. What is the output?

```
int a=5;
printf("%d",a++);
```

A) 6
B) 5
C) 4
D) Error

■ Correct Answer: B) 5

Explanation: Post-increment returns original value (5), then increments.

Q197. What is the output?

```
int a=5;
printf("%d",++a);
```

A) 6
B) 5
C) 4
D) Error

■ Correct Answer: A) 6

Explanation: Pre-increment increments first (to 6), then returns 6.

Q198. What is the output?

```
int a=5;
printf("%d",a--);
```

A) 5
B) 4
C) 6
D) Error

■ Correct Answer: A) 5

Explanation: Post-decrement returns original value (5), then decrements.

Q199. What is the output?

```
int a=5;
printf("%d",--a);
```

A) 5
B) 4
C) 6
D) Error

■ Correct Answer: B) 4

Explanation: Pre-decrement decrements first (to 4), then returns 4.

Q200. What is the output?

```
int a=5;
a++;
printf("%d",a);
```

A) 5
B) 6
C) 7
D) 4

■ Correct Answer: B) 6

Explanation: a++ increments a to 6; printf shows 6.

Q201. What is the output?

```
int a=5, b;
b=a++;
printf("%d %d",a,b);
```

A) 5 5
B) 6 5
C) 5 6
D) 6 6

■ Correct Answer: B) 6 5

Explanation: b=a++ assigns 5 to b (post), then a becomes 6.

Q202. What is the output?

```
int a=5, b;
b=++a;
printf("%d %d",a,b);
```

A) 5 5
B) 6 6
C) 5 6
D) 6 5

■ Correct Answer: B) 6 6

Explanation: ++a increments a to 6, then assigns 6 to b.

Q203. What is the output?

```
int x=3;
printf("%d", x++ + ++x);
```

A) 8
B) 7
C) Undefined
D) 6

■ Correct Answer: C) Undefined

Explanation: Modifying x twice in same expression without sequence point = UB.

Q204. ++ and -- are _____ operators.

A) Binary
B) Ternary
C) Unary
D) Bitwise

■ Correct Answer: C) Unary
Explanation: They operate on a single operand.

Q205. Which has higher precedence: i++ or ++i in isolation?

A) i++
B) ++i
C) Both same
D) Depends on context

■ Correct Answer: A) i++
Explanation: Post-increment has higher precedence than pre-increment.

Q206. What is the output?

```
int a=1,b=1;
a=b++;
printf("%d %d",a,b);
```

A) 1 1
B) 2 2
C) 1 2
D) 2 1

■ Correct Answer: C) 1 2
Explanation: a=b++ assigns 1 to a (post-increment), then b becomes 2.

Q207. What is the output?

```
int i=5;
printf("%d",i+++1);
```

A) 6
B) 7
C) Error
D) 11

■ Correct Answer: A) 6
Explanation: Parsed as (i++)+1: returns 5+1=6 (i becomes 6 afterwards).

Q208. What is the output?

```
int a=10;
printf("%d %d",a--,a);
```

A) 10 9
B) 9 10
C) Undefined
D) 10 10

■ Correct Answer: C) Undefined

Explanation: Order of evaluation of function arguments is unspecified; undefined behaviour.

Q209. What is the output?

```
int a=5;
while(a-->0)
printf("%d ",a);
```

A) 5 4 3 2 1
B) 4 3 2 1 0
C) 5 4 3 2 1 0
D) Error

■ Correct Answer: B) 4 3 2 1 0

Explanation: a-->0 is (a--)>0: uses a then decrements. Loop runs while original a>0; prints decremented a.

Q210. The 'goes to' operator --> in C is actually:

A) A special operator
B) Post-decrement followed by greater-than
C) Pre-decrement followed by greater-than
D) Error

■ Correct Answer: B) Post-decrement followed by greater-than

Explanation: a-->0 parses as (a--)>0; a post-decremented, compared to 0.

Q211. What is the output?

```
int x=0;
printf("%d",x++==0);
```

A) 0
B) 1
C) Error
D) Undefined

■ Correct Answer: B) 1

Explanation: x++ returns 0; 0==0 is true (1).

Q212. What is the output?

```
int a=5;
printf("%d",a++++); // invalid?
```

A) 5
B) 6
C) Compile error
D) 7

> ■ **Correct Answer: C) Compile error**
> **Explanation:** a++++ is parsed as (a++)++ which is invalid; cannot apply ++ to an rvalue.

Q213. What is the output?

```
int a=5;
int b = -a++;
printf("%d %d",a,b);
```

A) 6 -5
B) 5 -5
C) 6 -6
D) 5 -6

> ■ **Correct Answer: A) 6 -5**
> **Explanation:** a++ returns 5; unary - gives -5; b=-5; then a increments to 6.

Q214. What is the output?

```
for(int i=0;i<3;i++)
printf("%d ",i);
```

A) 0 1 2
B) 1 2 3
C) 0 1 2 3
D) Error

> ■ **Correct Answer: A) 0 1 2**
> **Explanation:** Loop runs for i=0,1,2; prints each before increment.

Q215. What is the output?

```
for(int i=3;i>0;--i)
printf("%d ",i);
```

A) 3 2 1
B) 2 1 0
C) 3 2 1 0
D) Error

> ■ **Correct Answer: A) 3 2 1**
> **Explanation:** Loop runs for i=3,2,1 (pre-decrement in update doesn't matter here).

Q216. What is the output?

```
int a=1;
printf("%d",a++*a++);
```

A) 1
B) 2
C) Undefined
D) 3

■ Correct Answer: C) Undefined

Explanation: Modifying a twice without sequence point = undefined behaviour.

Q217. Pre-increment vs Post-increment in a standalone statement: int a=5; a++;

A) a becomes 6 with a++
B) a becomes 6 with ++a
C) Both increment a to 6
D) Neither changes a

■ Correct Answer: C) Both increment a to 6

Explanation: In standalone statements both ++a and a++ have same effect.

Q218. What is the output?

```
int a=5;
printf("%d %d",++a,a++);
```

A) 6 5
B) 6 6
C) Undefined
D) 7 5

■ Correct Answer: C) Undefined

Explanation: Unspecified order of evaluation of function arguments; undefined behaviour.

Q219. What is the output?

```
int x=10;
x++; ++x;
printf("%d",x);
```

A) 10
B) 11
C) 12
D) 13

■ Correct Answer: C) 12

Explanation: x++=11, ++x=12; both standalone increments.

Q220. What is the output?

```
int a=5;
if(a++>5)
printf("Yes");
else
printf("No");
```

A) Yes
B) No
C) Error
D) Nothing

> ■ **Correct Answer: B) No**
> **Explanation:** a++ uses 5 in comparison; 5>5 is false; else branch runs.

Q221. What is the output?

```
int a=5;
if(++a>5)
printf("Yes");
else
printf("No");
```

A) Yes
B) No
C) Error
D) Nothing

> ■ **Correct Answer: A) Yes**
> **Explanation:** ++a increments to 6 before comparison; 6>5 is true.

Q222. What is the output?

```
int a=5;
printf("%d",--a + a--);
```

A) 9
B) 8
C) Undefined
D) 10

> ■ **Correct Answer: C) Undefined**
> **Explanation:** Modifying a twice in same expression = undefined behaviour.

Q223. What is the output?

```
int a=5;
int b=a-- - --a;
printf("%d",b);
```

A) 2
B) 1
C) Undefined
D) 0

■ Correct Answer: C) Undefined
Explanation: Multiple modifications of a without sequence point = UB.

Q224. What is the output?

```
int i=0;
printf("%d",i++);
```

A) 0
B) 1
C) Error
D) -1

■ Correct Answer: A) 0
Explanation: Post-increment returns 0 (original), then increments.

Q225. Which of these has the HIGHEST precedence?

A) `++ (post)`
B) `++ (pre)`
C) Unary -
D) !

■ Correct Answer: A) ++ (post)
Explanation: Postfix operators (including post-++) have the highest precedence.

Section 7: Conditional (Ternary) Operator

Q226. What is the syntax of the conditional operator in C?

A) condition ? expr1 : expr2
B) `if(condition) expr1 else expr2`
C) condition : expr1 ? expr2
D) condition ?? expr1 : expr2

■ Correct Answer: A) condition ? expr1 : expr2
Explanation: The ternary/conditional operator syntax: condition ? expr_if_true : expr_if_false.

Q227. What is the output?

```
printf("%d", 5>3 ? 10 : 20);
```

A) 10
B) 20
C) 5
D) 3

■ Correct Answer: A) 10
Explanation: 5>3 is true; expression evaluates to 10.

Q228. What is the output?

```
printf("%d", 5<3 ? 10 : 20);
```

A) 10
B) 20
C) 5
D) 3

■ Correct Answer: B) 20
Explanation: 5<3 is false; expression evaluates to 20.

Q229. The conditional operator ?: is _____ operator.

A) Unary
B) Binary
C) Ternary
D) Bitwise

■ Correct Answer: C) Ternary
Explanation: ?: takes three operands; it is the only ternary operator in C.

Q230. What is the output?

```
int a=5,b=10;
printf("%d", a>b ? a : b);
```

A) 5
B) 10
C) 15
D) 0

■ Correct Answer: B) 10
Explanation: a>b is false; returns b=10.

Q231. What is the output?

```
int a=5,b=5;
printf("%d", a==b ? 1 : 0);
```

A) 0
B) 1
C) 5
D) Error

■ Correct Answer: B) 1

Explanation: a==b is true; returns 1.

Q232. What is the output?

```
int x=0;
printf("%d", x ? 100 : 200);
```

A) 100
B) 200
C) 0
D) Error

■ Correct Answer: B) 200

Explanation: x=0 is false; returns 200.

Q233. Conditional operator can be nested like:

```
int x=5;
printf("%d", x>10?1: x>0?2:3);
```

A) 1
B) 2
C) 3
D) Error

■ Correct Answer: B) 2

Explanation: x>10 false; x>0 true; returns 2.

Q234. What is the output?

```
int a=-5;
printf("%d", a<0 ? -a : a);
```

A) -5
B) 5
C) 0
D) Error

■ Correct Answer: B) 5

Explanation: a<0 is true; returns -a = -(-5) = 5 (absolute value pattern).

Q235. Which is TRUE about the conditional operator?

A) It can replace any if-else
B) It is right-associative
C) It always returns int
D) It cannot be nested

■ Correct Answer: B) It is right-associative

Explanation: ?: is right-associative when nested.

Q236. What is the output?

```
int x=10,y=20;
int z = x>y ? x++ : y++;
printf("%d %d %d",x,y,z);
```

A) 10 21 20
B) 11 20 10
C) 10 21 21
D) 10 20 20

■ Correct Answer: A) 10 21 20

Explanation: x>y false; y++ executed (z=20, y becomes 21); x unchanged.

Q237. What is the output?

```
printf("%d", 1 ? 2 ? 3 : 4 : 5);
```

A) 3
B) 4
C) 5
D) 2

■ Correct Answer: A) 3

Explanation: Outer: 1 is true, evaluate left. Inner: 2 is true, returns 3.

Q238. What is the output?

```
printf("%d", 0 ? 2 ? 3 : 4 : 5);
```

A) 3
B) 4
C) 5
D) 2

■ Correct Answer: C) 5

Explanation: Outer: 0 is false, returns 5.

Q239. What is the output?

```
int a=5;
a = a>0 ? a*2 : a/2;
printf("%d",a);
```

A) 10
B) 2
C) 5
D) 0

■ Correct Answer: A) 10

Explanation: a>0 true; a=5*2=10.

Q240. Can the conditional operator be used as an lvalue in C?

A) Yes
B) No
C) Only in C99
D) Only with pointers

■ Correct Answer: B) No

Explanation: In C, the result of ?: cannot be assigned to (not an lvalue).

Q241. What is the output?

```
int max(int a,int b){return a>b?a:b;}
printf("%d",max(3,7));
```

A) 3
B) 7
C) 10
D) Error

■ Correct Answer: B) 7

Explanation: 3>7 false; returns b=7.

Q242. What is the output?

```
printf("%s", 5%2==0 ? "Even" : "Odd");
```

A) Even
B) Odd
C) Error
D) 5

■ Correct Answer: B) Odd

Explanation: 5%2=1 != 0; false; returns "Odd".

Q243. What is the output?

```
printf("%s", 10%2==0 ? "Even" : "Odd");
```

A) Even
B) Odd
C) Error
D) 10

■ Correct Answer: A) Even
Explanation: 10%2=0 == 0; true; returns "Even".

Q244. What is the output?

```
int a=5,b=10,c=15;
printf("%d", a<b ? (b<c?c:b) : a);
```

A) 15
B) 10
C) 5
D) Error

■ Correct Answer: A) 15
Explanation: a<b true; inner: b<c true; returns c=15.

Q245. What is the output?

```
int x=5;
printf("%d", x>0 ? x>10 ? 1 : 2 : 3);
```

A) 1
B) 2
C) 3
D) Error

■ Correct Answer: B) 2
Explanation: x>0 true; x>10 false; returns 2.

Q246. The conditional operator has _____ precedence than assignment.

A) Lower
B) Higher
C) Equal
D) Undefined

■ Correct Answer: B) Higher
Explanation: ?: has higher precedence than = (assignment).

Q247. What is the output?

```
int a=0;
printf("%d", a=5, a>0 ? 1 : 0);
```

A) 1
B) 0
C) 5
D) Undefined

■ Correct Answer: A) 1

Explanation: printf with multiple args: format uses only first arg after format; a=5 executes, then a>0?1:0=1 is the second arg which is printed.

Q248. What is the output?

```
int x=5,y=10;
printf("%d", x<y ? y-x : x-y);
```

A) 5
B) -5
C) 15
D) 10

■ Correct Answer: A) 5

Explanation: x<y true; y-x=10-5=5.

Q249. Which of these is EQUIVALENT to: int abs_a = (a>=0)?a:-a;?

A) `if(a>=0)abs_a=a else abs_a=-a;`
B) `if(a>=0){abs_a=a;} else{abs_a=-a;}`
C) Both A and B
D) None

■ Correct Answer: B) if(a>=0){abs_a=a;} else{abs_a=-a;}

Explanation: Proper C if-else syntax in option B is equivalent.

Q250. What is the output?

```
int a=10,b=20;
printf("%d", a<b ? a+=5 : b+=5);
printf(" %d",a);
```

A) 15 15
B) 10 10
C) 20 10
D) Undefined

■ Correct Answer: A) 15 15

Explanation: a<b true; a+=5 executes; a becomes 15; printf prints 15, then a=15.

Section 8: sizeof Operator

Q251. sizeof is a _____ operator in C.

A) Function
B) Macro
C) Compile-time operator
D) Runtime function

■ Correct Answer: C) Compile-time operator
Explanation: sizeof is evaluated at compile-time (for most cases) as a compile-time operator.

Q252. What does sizeof return?

A) Number of elements
B) Size in bits
C) Size in bytes
D) Pointer value

■ Correct Answer: C) Size in bytes
Explanation: sizeof returns the size of a type or expression in bytes.

Q253. What is the type of value returned by sizeof?

A) int
B) unsigned int
C) size_t
D) long

■ Correct Answer: C) size_t
Explanation: sizeof returns a value of type size_t (an unsigned integer type).

Q254. What is sizeof(char) guaranteed to be in C?

A) 1
B) 2
C) 4
D) Depends on platform

■ Correct Answer: A) 1
Explanation: sizeof(char) is always 1 byte by definition in C.

Q255. What is sizeof(int) on a typical 32-bit system?

A) 1
B) 2
C) 4
D) 8

■ Correct Answer: C) 4
Explanation: On most 32-bit and 64-bit systems, sizeof(int) = 4 bytes.

Q256. What is sizeof(double) typically?

A) 4
B) 8
C) 2
D) 16

■ Correct Answer: B) 8
Explanation: sizeof(double) is typically 8 bytes.

Q257. What is the output?

```
printf("%zu", sizeof(int));
```

A) 1
B) 2
C) 4
D) Platform dependent

■ Correct Answer: D) Platform dependent
Explanation: %zu prints size_t; value is platform-dependent (commonly 4).

Q258. What is the output?

```
char arr[10];
printf("%zu", sizeof(arr));
```

A) 1
B) 10
C) 4
D) Pointer size

■ Correct Answer: B) 10
Explanation: sizeof of an array gives total bytes: 10 chars * 1 byte = 10.

Q259. What is the output?

```
int arr[5];
printf("%zu", sizeof(arr));
```

A) 5
B) 10
C) 20
D) Platform dependent

■ Correct Answer: C) 20

Explanation: 5 ints * 4 bytes each = 20 bytes (on 32/64-bit with 4-byte int).

Q260. How to find number of elements in array arr using sizeof?

A) `sizeof(arr)`
B) `sizeof(arr)/sizeof(arr[0])`
C) `sizeof(arr[0])`
D) `sizeof(*arr)`

■ Correct Answer: B) sizeof(arr)/sizeof(arr[0])

Explanation: Divide total array size by size of one element.

Q261. What is the output?

```
int a=10;
printf("%zu",sizeof(a));
```

A) 10
B) 4
C) 1
D) 2

■ Correct Answer: B) 4

Explanation: sizeof an int variable is 4 bytes typically.

Q262. sizeof can be applied to:

A) Types only
B) Variables only
C) Both types and expressions
D) Functions

■ Correct Answer: C) Both types and expressions

Explanation: sizeof works with types (sizeof(int)) and expressions (sizeof(a)).

Q263. What is sizeof(float) typically?

A) 8
B) 4
C) 2
D) 16

> ■ **Correct Answer: B) 4**
> **Explanation:** float is typically 4 bytes (single precision).

Q264. What is sizeof(long double) typically?

A) 8
B) 10 or 12 or 16
C) 4
D) 1

> ■ **Correct Answer: B) 10 or 12 or 16**
> **Explanation:** long double is 10, 12, or 16 bytes depending on platform/compiler.

Q265. What is the output?

```
printf("%zu", sizeof(5+3.0));
```

A) 4
B) 8
C) 2
D) 1

> ■ **Correct Answer: B) 8**
> **Explanation:** 5+3.0 promotes to double; sizeof(double)=8.

Q266. Does sizeof evaluate its operand expression?

A) Yes, always
B) No, it does not evaluate
C) Only if it contains function calls
D) Only in C99

> ■ **Correct Answer: B) No, it does not evaluate**
> **Explanation:** sizeof does NOT evaluate the operand; it only determines its type.

Q267. What is the output?

```
int a=5;
printf("%zu", sizeof(a++));
printf(" %d",a);
```

A) 4 6
B) 4 5
C) 1 5
D) Error

■ Correct Answer: B) 4 5
Explanation: sizeof does not evaluate a++; a remains 5.

Q268. What is sizeof(void) in C?

A) 1
B) 0
C) 4
D) Undefined / implementation-defined

■ Correct Answer: D) Undefined / implementation-defined
Explanation: sizeof(void) is not defined in standard C (undefined behaviour).

Q269. What is sizeof(int *) on a 64-bit system?

A) 4
B) 8
C) 2
D) Depends on int

■ Correct Answer: B) 8
Explanation: On 64-bit systems, all pointers are typically 8 bytes.

Q270. What is sizeof(int *) on a 32-bit system?

A) 4
B) 8
C) 2
D) Depends on int

■ Correct Answer: A) 4
Explanation: On 32-bit systems, all pointers are typically 4 bytes.

Q271. What is the output?

```
struct{int a; char b;} s;
printf("%zu",sizeof(s));
```

A) 5
B) 8
C) Platform/alignment dependent
D) 4

■ Correct Answer: C) Platform/alignment dependent

Explanation: Struct size depends on padding/alignment; commonly 8 on 4-byte aligned systems.

Q272. sizeof can be used without parentheses when:

A) Always
B) Never
C) When operand is a variable/expression
D) When operand is a type

■ Correct Answer: C) When operand is a variable/expression

Explanation: sizeof var (without parens) is valid for expressions; sizeof(type) requires parens.

Q273. What is the output?

```
printf("%zu",sizeof "Hello");
```

A) 5
B) 6
C) 4
D) Error

■ Correct Answer: B) 6

Explanation: "Hello" is a char array of 6 elements (5 chars + null terminator); sizeof=6.

Q274. The correct format specifier for printing sizeof result is:

A) %d
B) %i
C) %zu
D) %u

■ Correct Answer: C) %zu

Explanation: %zu is the correct specifier for size_t (z=size_t modifier, u=unsigned).

Q275. What is the value of sizeof(sizeof(int)) on all platforms?

A) 4
B) `sizeof(size_t)`
C) 8
D) Depends on int

> ■ **Correct Answer: B) sizeof(size_t)**
> **Explanation:** sizeof(int) returns size_t; sizeof(size_t) is typically 4 or 8 depending on platform.

ANSWER KEY — All 275 Questions

Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans	Q#	Ans
1	A	2	B	3	B	4	B	5	B	6	A	7	A	8	B
9	B	10	B	11	B	12	B	13	A	14	B	15	C	16	A
17	B	18	C	19	A	20	C	21	B	22	A	23	B	24	A
25	B	26	C	27	A	28	B	29	B	30	A	31	C	32	C
33	B	34	B	35	A	36	B	37	B	38	D	39	B	40	B
41	B	42	A	43	B	44	B	45	A	46	B	47	B	48	C
49	B	50	B	51	A	52	B	53	B	54	B	55	B	56	A
57	B	58	B	59	A	60	B	61	B	62	B	63	B	64	B
65	A	66	B	67	B	68	C	69	A	70	B	71	C	72	B
73	B	74	B	75	A	76	B	77	B	78	B	79	B	80	B
81	B	82	A	83	B	84	B	85	A	86	B	87	B	88	B
89	B	90	B	91	A	92	A	93	B	94	B	95	B	96	C
97	B	98	B	99	B	100	A	101	B	102	B	103	A	104	B
105	A	106	B	107	B	108	B	109	C	110	C	111	A	112	B
113	B	114	A	115	C	116	B	117	B	118	A	119	B	120	A
121	B	122	B	123	B	124	C	125	B	126	B	127	B	128	A
129	B	130	C	131	A	132	B	133	B	134	C	135	A	136	B
137	B	138	B	139	A	140	C	141	A	142	D	143	C	144	A
145	C	146	B	147	B	148	C	149	A	150	B	151	B	152	A
153	B	154	B	155	A	156	B	157	A	158	B	159	A	160	B
161	A	162	B	163	B	164	A	165	C	166	A	167	B	168	B
169	C	170	C	171	A	172	C	173	B	174	A	175	B	176	B
177	B	178	C	179	B	180	A	181	B	182	C	183	B	184	B
185	B	186	B	187	C	188	B	189	B	190	A	191	B	192	A
193	B	194	A	195	A	196	B	197	A	198	A	199	B	200	B
201	B	202	B	203	C	204	C	205	A	206	C	207	A	208	C
209	B	210	B	211	B	212	C	213	A	214	A	215	A	216	C

217	C	218	C	219	C	220	B	221	A	222	C	223	C	224	A
225	A	226	A	227	A	228	B	229	C	230	B	231	B	232	B
233	B	234	B	235	B	236	A	237	A	238	C	239	A	240	B
241	B	242	B	243	A	244	A	245	B	246	B	247	A	248	A
249	B	250	A	251	C	252	C	253	C	254	A	255	C	256	B
257	D	258	B	259	C	260	B	261	B	262	C	263	B	264	B
265	B	266	B	267	B	268	D	269	B	270	A	271	C	272	C
273	B	274	C	275	B										

Note: *Questions involving undefined behaviour in C are intentionally included to build awareness of common pitfalls tested in competitive and academic exams. Always compile and run code to verify output when in doubt. Good luck with your OAVS TGT Computer Science Examination!*